

Documento de Justificativa Técnica

Aluno: Pedro Leal Murad

RM: 565460

Turma: 2TIAPF 2026

Disciplina: Generative AI Advanced Net

Escolha do banco vetorial

O banco vetorial escolhido foi o Qdrant, executado localmente em contêiner Docker. A escolha foi feita porque o Qdrant é uma solução específica para armazenamento e busca de vetores, possui integração direta com Python e permite trabalhar com metadados associados aos trechos indexados. Isso é importante para o fluxo RAG, pois não basta recuperar um vetor parecido: também é necessário saber qual documento e qual trecho originaram aquela informação.

Além disso, o uso via Docker facilita a execução da solução em ambiente local, sem depender de serviço pago em nuvem. Para a proposta acadêmica, essa abordagem torna a demonstração mais simples e reproduzível.

Escolha do modelo de linguagem

Foi utilizada a API da OpenAI, com o modelo gpt-4o-mini para geração de respostas e o modelo text-embedding-3-small para embeddings. A escolha foi baseada na boa qualidade de resposta, baixo custo relativo e compatibilidade com aplicações RAG.

O modelo de embedding utilizado gera vetores de dimensão 1536, e essa mesma dimensão foi configurada na coleção do Qdrant. Essa compatibilidade é essencial para evitar erro na indexação e na busca semântica.

As credenciais e parâmetros sensíveis foram mantidos em variáveis de ambiente, por meio do arquivo .env, evitando exposição de chaves diretamente no código.

Melhorias de design implementadas

A primeira melhoria foi a separação de responsabilidades. O projeto foi dividido em rotas, serviços, modelos, configurações e utilidades. Dessa forma, as rotas HTTP não concentram regra de negócio; elas apenas recebem a requisição e delegam o processamento para os serviços adequados.

A segunda melhoria foi o isolamento dos serviços externos. A integração com OpenAI ficou concentrada no serviço de LLM, enquanto a comunicação com o Qdrant ficou em um serviço próprio. Essa decisão facilita manutenção e permite trocar futuramente o provedor de modelo ou o banco vetorial com menor impacto no restante do sistema.

A terceira melhoria foi a estratégia de chunking com metadados. Os documentos são divididos em trechos menores com sobreposição, e cada trecho recebe informações como fonte, índice e tamanho. Isso melhora a recuperação semântica e permite rastrear as fontes usadas na resposta.

Também foram adicionados logging estruturado, tratamento de erros nas rotas principais, proteção básica por X-API-Key, arquivos Docker para automação da execução e testes automatizados básicos. Essas decisões deixam o back-end mais organizado, mais fácil de avaliar e mais próximo de uma aplicação real.

Conclusão

A solução entrega um fluxo RAG funcional: documentos são indexados no Qdrant, perguntas são transformadas em embeddings, trechos relevantes são recuperados semanticamente e o modelo de linguagem gera uma resposta com base no contexto encontrado. O projeto mantém compatibilidade com o front-end e apresenta melhorias arquiteturais alinhadas aos requisitos da atividade.